

PORTFOLIO

JAVA · SPRING BOOT BACKEND DEVELOPER

신재화

EMAIL

jaehwa06@naver.com

GITHUB

github.com/necteo

BLOG

necteo.github.io

목차

01	자기소개 개인 정보 · 학력 · 교육 · 자격증 · 강점	P. 02
02	기술 스택 Backend · Frontend · 실시간 · DevOps · DB	P. 03 - 04
03	프로젝트 한눈에 보기 AllCamp · AllMovie · BookTalk	P. 05
04	AllCamp JSP 팀프로젝트 · 커뮤니티/게시판 · 계층형 댓글	P. 06 - 13
05	AllMovie Spring Boot · 실시간 WebSocket · 결제 · CI/CD	P. 14 - 24
06	BookTalk Spring Boot + React · AI 챗봇 · OAuth2 · 개인 프로젝트	P. 25 - 34

자기소개 & 성장 여정

PERSONAL

Name 신 재 화 / Shin Jaehwa
Email jaehwa06@naver.com
GitHub github.com/necteo
Blog necteo.github.io

EDUCATION & CERTIFICATES

학부 국립금오공과대학교 · 컴퓨터소프트웨어공학과 3.89/4.5 · 2024.02 졸업
교육 쌍용강북교육센터 · AWS & CI/CD 기반 JAVA 클라우드 Full-Stack 개발자 양성과정
140일 · 2025.07 — 2026.02
자격증 SQLD · 정보처리기사 · 컴활 1급
수상 코더스아이티 제1회 프로그래밍 대회 2023.09

GROWTH TIMELINE

- 2017.03 — 2024.02
국립금오공과대학교
컴퓨터소프트웨어공학 학사 졸업
- 2025.07 — 2026.02
쌍용강북 Java 클라우드 Full-Stack 과정
AWS · CI/CD 기반 풀스택 개발 · 140일
- 2025.10 — 2025.11
AllCamp · JSP 팀프로젝트
커뮤니티 / 게시판 담당
- 2025.12 — 2026.01
AllMovie · Spring Boot 팀프로젝트
영화 조회 · 매점 · 결제 · 실시간 · CI/CD 담당
- 2026.02 — 2026.06
BookTalk · 개인 프로젝트
Spring Boot + React + AI 풀스택

기술 스택 — Backend

Java

- 제네릭으로 목록 응답 타입과 페이지네이션 계산을 공통화
- Enum에 메시지와 HTTP 상태를 담아 예외를 타입으로 정의
- record로 불변 응답 DTO를 만들어 Entity를 계층 밖으로 내보내지 않도록 분리
- Optional · Stream으로 조회 결과의 부재와 변환을 명시적으로 처리

Spring Data JPA / MyBatis

- resultMap의 association · collection으로 주문 조회 N+1 제거
- 동적 쿼리를 활용해 전체 · 제목 · 작성자 · 내용별 조건 검색 구현
- @SelectKey로 시퀀스 생성 키를 받아 연관 INSERT에 연결
- JPA Repository 기반 CRUD와 변경 감지로 수정 반영

Spring Boot / MVC

- 전역 예외 핸들러로 예외 처리 지점과 응답 형식을 한 곳으로 통일
- AllCamp JSP → AllMovie JSP + Vue.js CDN → BookTalk React SPA 순으로 View 계층 발전
- RESTful API 설계 · DTO · VO 패턴으로 계층 간 데이터 전달

Spring Security

- JdbcUserDetailsManager + BCrypt로 DB 기반 인증 구성
- USER · MANAGER · ADMIN 역할 기반 접근 제어
- Access 15분 무상태 검증 + Refresh 7일 DB 저장 · 회전
- OAuth 소셜 로그인 Google · Kakao · Naver 3종 연동

기술 스택 — Frontend · 실시간 · Infra

TypeScript / React

- 함수형 컴포넌트와 Hooks · React Router 기반 SPA 구성
- API 응답 타입을 공통 파일에 정의하고 useQuery 제네릭으로 지정해 응답 구조 고정
- TanStack Query로 서버 상태를 캐싱하고 변경 후 refetch로 목록 갱신
- Axios 인터셉터로 401 감지 후 토큰 자동 재발급 · 재발급 요청은 재시도 제외

Database

- 단품 · 콤보를 한 테이블에 두고 구분 플래그로 나눠 메뉴 추가 시 데이터 등록만으로 동작
- 좋아요를 대상 종류와 대상 번호를 가진 단일 테이블로 설계해 게시글 · 댓글에 함께 사용
- Oracle OFFSET FETCH로 목록 페이징 처리 · 계층형 쿼리로 대댓글이 들어갈 위치 계산
- JPA ddl-auto로 MySQL 스키마 생성 · 엔티티 기준 테이블 설계

CI/CD Pipeline

- Jenkins 파이프라인으로 배포 자동화 · GitHub Actions로도 같은 파이프라인 구성
- AWS EC2 SSH 원격 자동 배포
- Checkout → Build → Docker → Push → Deploy
- Nginx 리버스 프록시와 Let's Encrypt로 HTTPS 적용

JavaScript / WebSocket

- DOM 조작과 Ajax 비동기 통신 구현
- JSP 안에 Vue.js CDN과 Pinia로 컴포넌트 · 상태 관리 적용
- STOMP.js + SockJS로 주문 발생 시 실시간 알림 수신
- 포트원(아임포트) 결제 라이브러리 연동

Docker

- Dockerfile로 Spring Boot 이미지 패키징
- docker-compose로 컨테이너 실행 · 포트 매핑
- DockerHub로 이미지 버전 관리

프로젝트 한눈에 보기

TEAM · 6 MEMBERS 01 / 03

AllCamp

캠핑을 좋아하는 사용자가 캠핑장을 예약하고, 주변 맛집·축제를 확인하고, 커뮤니티에서 후기를 나누는 통합 플랫폼입니다.

TYPE	팀 프로젝트
PERIOD	2025.10.13 — 2025.11.20
ROLE	공지사항 · Q&A 게시판 · 댓글/대댓글 · 좋아요

Java JSP MyBatis Oracle
jQuery

TEAM · 3 MEMBERS 02 / 03

AllMovie

영화관의 핵심 업무 흐름을 디지털화한 플랫폼입니다. 관객은 영화 예매와 매점 주문을 한 번에 하고, 직원은 주문 현황을 실시간으로 받습니다.

TYPE	팀 프로젝트
PERIOD	2025.12.22 — 2026.01.30
ROLE	영화 조회 · 매점 · 주문 · 결제 · 실시간 · 인증 · CI/CD

Spring Boot Vue.js WebSocket
Docker Jenkins

PERSONAL 03 / 03

BookTalk

책을 좋아하는 사용자가 도서 정보를 탐색하고, 게시판에서 소통하며, AI 챗봇에게 책을 추천받는 풀스택 SPA입니다.

TYPE	개인 프로젝트
PERIOD	2026.02.12 — 2026.06.14
ROLE	기획 · 설계 · 개발 전 과정

Spring Boot React 19 JWT
OAuth2 Spring AI

프로젝트 개요

PROJECT 01 · TEAM

AllCamp

캠핑을 좋아하는 사용자가 캠핑장을 예약하고, 주변 맛집·축제를 확인하고, 커뮤니티에서 후기를 나누는 통합 플랫폼입니다. 저는 공지사항과 Q&A 게시판, 댓글과 좋아요를 담당했습니다.

담당 기능

- 공지사항 · Q&A 게시판 CRUD (검색 · 페이징)
- 계층형 댓글 / 대댓글 작성 · 수정 · 삭제
- DAO · Model · VO 레이어 직접 설계 · 구현
- 좋아요 기능 구현
- WAR Export 후 AWS EC2에 직접 배포

PROJECT
TYPE

팀 프로젝트 (6인)

PERIOD

2025.10.13 — 2025.11.20 (약 6주)

MY ROLE

공지사항 · Q&A 게시판 · 댓글/대댓글 · 좋아요

GITHUB

github.com/JSPWebProject/AllCamp

TECH STACK

Java

JSP

Servlet

MyBatis

Oracle

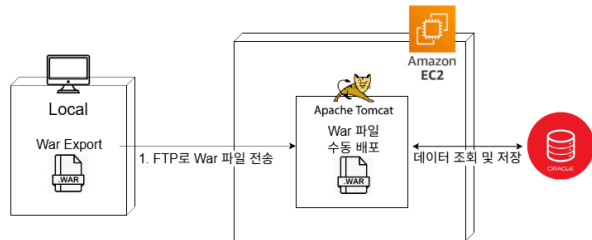
jQuery

Ajax

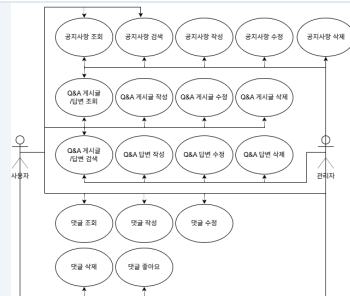
AWS EC2

Tomcat

설계 산출물 & 배포 아키텍처



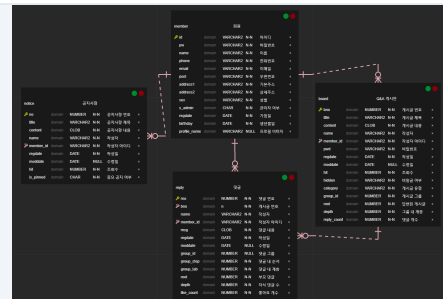
01 배포 아키텍처



02 UC 다이어그램



03 순서도



04 ER 다이어그램

개발 목표 & 성과

개발 목표

SQL CRUD와 MyBatis를 활용해 데이터 접근 계층을 구현한다.

MVC 패턴을 적용해 Model-DAO-VO 계층을 분리한다.

화면과 서버가 주고받는 데이터 구조를 일관되게 유지한다.

jQuery · Ajax로 페이지 이동 없이 화면을 동적으로 갱신한다.

AWS를 통해 웹 서비스를 배포한다.

성 과

MyBatis Mapper와 동적 쿼리를 활용해 SQL과 Java 코드를 분리하고, CRUD 기반 게시판 기능을 구현했습니다.

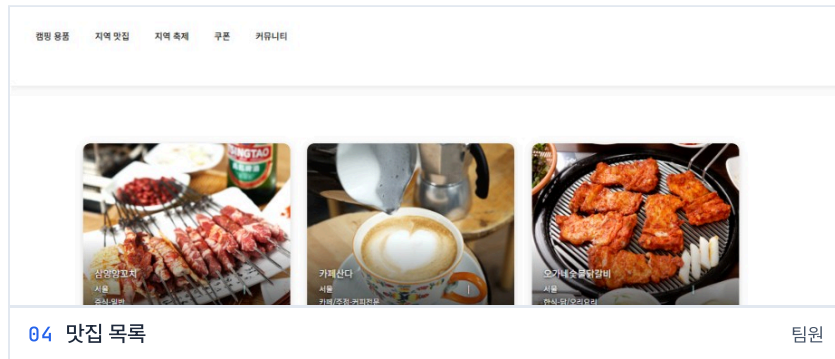
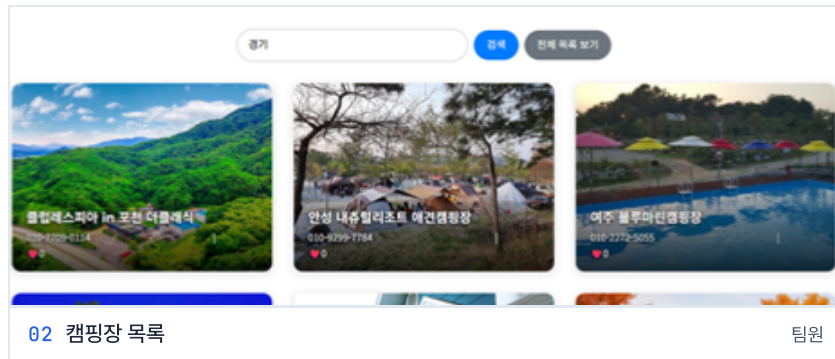
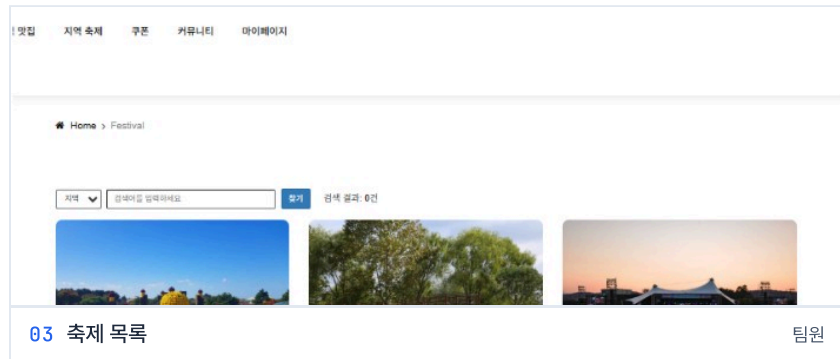
Model-DAO-VO 계층을 분리해 비즈니스 로직과 데이터 접근 로직의 의존성을 낮췄습니다.

화면 폼의 name과 VO 필드명을 같게 맞춰 같은 이름으로 주고받았습니다.

좋아요와 게시글 삭제를 Ajax로 처리해 페이지 전환 없이 반영했습니다.

AWS EC2에 Tomcat을 올려 외부에서 접속할 수 있게 배포했습니다.

프로젝트 둘러보기



담당 기능 — 공지사항

요구사항 분석표

기능	요소	연결 페이지	동작	우선순위
공지사항 목록 조회	번호, 제목, 작성자, 작성일, 조회수, 총 게시글 수, 페이지네이션	상세 조회, 작성, 검색	목록: 총 게시글 수 출력 · 10개 페이지 · 최근순 · 중요 공지 최상단 · 제목 클릭 시 상세	상
공지사항 상세 조회	제목, 작성자, 내용, 등록일시, 수정일시, 조회수, 수정·삭제·목록 버튼	목록 조회	상세 출력 · 조회수 증가 · 관리자만 수정·삭제	상
공지사항 작성	제목, 작성자, 내용, 비밀번호, 글쓰기·취소, 중요 공지 체크박스	상세 조회	작성 · 제목·내용 필수 · 작성자는 관리자 · 작성일 기본값은 글쓰기 시각 · 중요 공지 지정	상
공지사항 수정	제목, 작성자, 내용, 비밀번호, 수정하기·취소, 중요 공지 체크박스	상세 조회	내용 수정 · 필수 입력 · 수정 일시 저장 · 중요 공지 지정	상
공지사항 삭제	상세 조회의 삭제 버튼	목록 조회	공지 삭제	상
공지사항 검색	검색 유형 선택 상자, 검색 창, 검색 버튼	목록 조회	기본값은 제목 · 유형은 제목·작성자·내용 · 검색어 없으면 전체 · 중요 공지 최상단	상

구현 방식

목록 / 검색

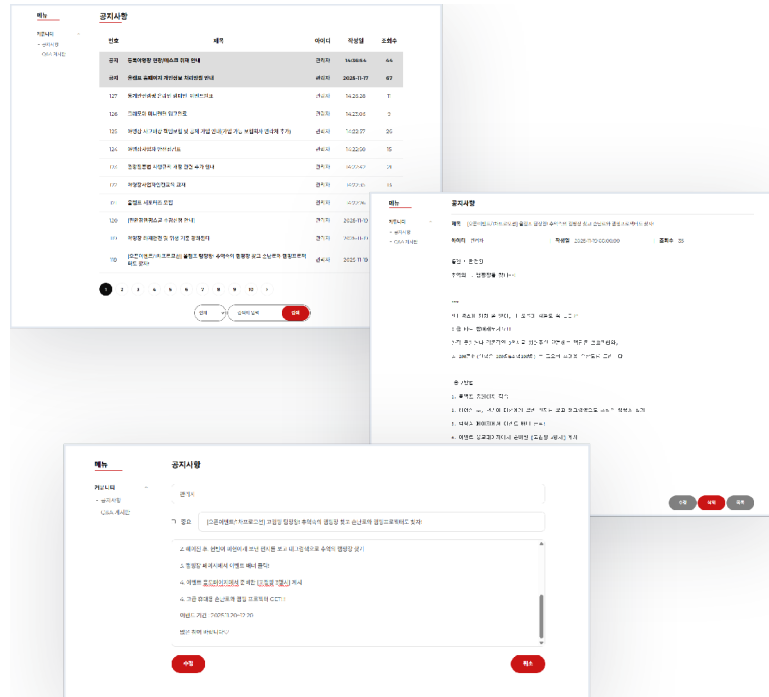
Oracle 페이징과 중요 공지 최상단 정렬을 적용하고, **MyBatis 동적 쿼리**로 제목·작성자·내용 다중 조건 검색을 구현했습니다.

상세 조회

공지 내용을 출력하고 조회수를 증가시킵니다.

작성 / 수정 / 삭제

관리자만 공지를 등록·수정·삭제할 수 있고, 중요 공지 여부를 지정할 수 있습니다.



담당 기능 — Q&A 게시판 (게시글 · 답변)

요구사항분석표

기능	요소	연결 페이지	동작	우선순위
게시글 목록 조회	게시글 유형, 번호, 제목, 작성자, 작성일, 조회수, 총 게시글 수, 페이지네이션	상세 조회, 작성, 검색	게시글-답변 출력 · 유형(전체/맛집/축제/캠핑용품/캠핑장) 분류 · 10개 페이지 · 최근순 · 비밀글 자동식	상
게시글 상세 조회	제목, 작성자, 작성일, 조회수, 내용, 수정-삭제-목록 버튼	목록 조회	게시글-답변 상세 출력 · 조회수 증가 · 비밀글은 작성자-관리자만 조회	상
게시글 작성	게시글 유형 선택 상자, 제목, 작성자, 내용, 비밀글 체크박스, 글쓰기-취소	상세 조회	작성 · 유형 필수 선택 · 필수 입력 · 비밀번호 검증 · 작성일 기본값은 글쓰기 시각	상
게시글 수정	게시글 유형 선택 상자, 제목, 작성자, 내용, 비밀글 체크박스, 수정-취소	상세 조회	수정 · 유형 필수 · 비밀번호 검증 · 수정일 저장	상
게시글 삭제	상세 조회의 삭제 버튼	목록 조회	삭제 · 비밀번호 검증	상
게시글 검색	검색 유형 선택 상자, 검색 창, 검색 버튼	목록 조회	게시글-답변 동시 검색 · 현재 유형에 맞는 목록 · 기본값은 제목 · 검색어 없으면 전체	상
답변 작성	제목, 작성자, 내용, 비밀번호, 글쓰기-취소	상세 조회	게시글에 답변 작성 · 대상 게시글 내용 미출력 · 필수 입력 · 비밀번호 검증 · 작성자 기본값은 관리자	상
답변 수정	제목, 작성자, 내용, 비밀번호, 글쓰기-취소	상세 조회	답변 수정 · 필수 입력 · 비밀번호 검증 · 수정일 저장	상
답변 삭제	상세 조회의 삭제 버튼	목록 조회	답변 삭제 · 비밀번호 검증	상

구현 방식

게시글 목록 / 검색

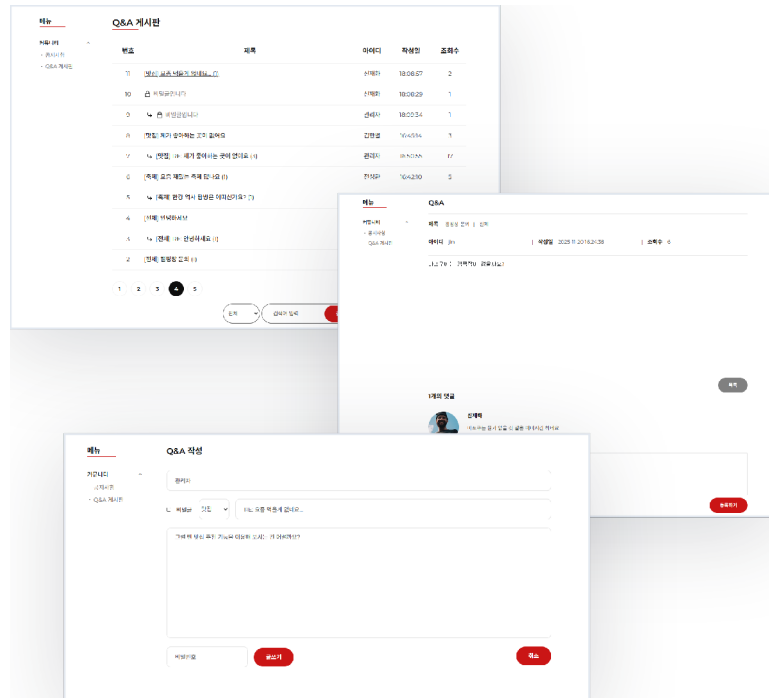
유형 분류와 페이징을 적용하고, **비밀글**은 목록에 자동식로 표시했습니다.

게시글 작성 / 수정 / 삭제

등록-수정-삭제할 때 **비밀번호**를 검증하고, 비밀글 설정을 넣었습니다.

답변 (계층 구조)

질문 글과 답변을 **group_id**로 묶고 **root-depth**로 종속 관계를 표현했습니다. 답변은 원글 아래 들여쓰기되어 정렬됩니다.



01 게시글 목록 02 게시글 상세 03 답변 작성

담당 기능 — 댓글 · 좋아요

요구사항분석표

기능	요소	연결 페이지	동작	우선순위
댓글 입력	댓글 입력창, 등록 버튼, 작성자, 작성일, 내용	게시글 상세 조회	게시글-답변에 댓글 작성 · 내용 필수 · 작성일 기본값은 댓글 작성 시각	중
댓글 조회	댓글 작성자, 프로필 이미지, 작성일, 내용, 좋아요 개수	게시글 상세 조회	상세에서 댓글 조회 · 대댓글은 상위보다 들어서 출력	중
댓글 수정	댓글 작성자, 프로필 이미지, 작성일, 내용, 좋아요 개수	게시글 상세 조회	수정 칸에 내용 출력 · 내용 필수 · 수정일 저장	중
댓글 삭제	댓글 작성자, 프로필 이미지, 작성일, 내용, 좋아요 개수	게시글 상세 조회	삭제 · 대댓글 있으면 내용만 '삭제된 댓글입니다'로 변경	중
댓글 좋아요	좋아요 여부, 댓글별 좋아요 총 개수	게시글 상세 조회	로그인 시 빨간 하트 · 비로그인/미선택 회색 하트 · 클릭 시 총 개수 증감	중하

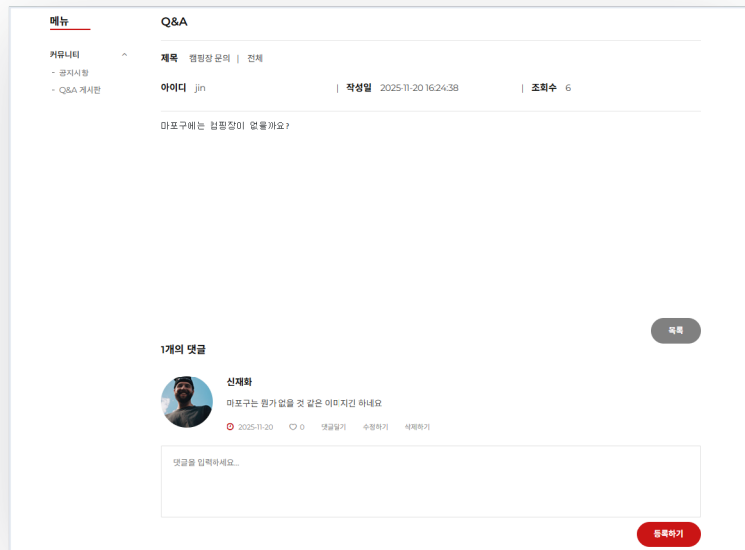
구현 방식

댓글 / 대댓글

댓글과 대댓글을 계층으로 출력하고, 삭제 시 하위 대댓글이 있으면 내용만 '삭제된 댓글입니다'로 변경했습니다.

댓글 좋아요

로그인한 사용자만 좋아요를 누를 수 있고, 같은 댓글에는 한 번만 반영됩니다.



대댓글 출력 순서 문제

⚠ 문제

시간순 정렬로 뒤집히는 대댓글 순서

댓글과 대댓글을 한 테이블에 쌓고 **작성 시간순으로 조회**하니, 대댓글이 부모 댓글 아래 붙지 않고 역순으로 출력되었습니다.

원인: 정렬 기준이 트리 구조를 무시하고 전역 시간으로만 적용됨

✅ 해결

CONNECT BY 서브트리 계산 + group_step 순서 관리

조회할 때 정렬하지 않고, **입력할 때 순서를 확정**하는 방식으로 바꿨습니다. **CONNECT BY**로 부모의 하위 댓글 수를 세고, 그만큼 뒤 댓글들의 **group_step**을 밀어 자리를 만든 뒤 저장합니다. 조회는 저장된 순서를 그대로 읽습니다.

```
-- 부모의 하위 댓글 수만큼 뒤 댓글의 순서를 밀어냄
UPDATE boardReply
SET group_step = group_step + 1
WHERE bno = #{bno}
  AND group_id = #{group_id}
  AND group_step >= #{group_step} + (
    SELECT COUNT(*) FROM boardReply
    START WITH no = #{no}
    CONNECT BY PRIOR no = root
);
```

프로젝트 개요

PROJECT 02 · TEAM

AllMovie

영화관의 핵심 업무 흐름을 디지털화한 플랫폼입니다. 관객은 영화 예매와 매점 주문을 한 번에 하고, 직원은 주문 현황을 실시간으로 받습니다. 저는 영화 조회 · 매점 · 주문 · 결제 · 실시간 통신 · 인증 · CI/CD를 담당했습니다.

담당 기능

- 메인 페이지 · 영화 목록 · 상세 조회
- 매점 상품 목록 · 상세 · 장바구니 · 주문 · 관리자 재고 관리
- 포트원 PG 결제 연동 · 결제 취소
- STOMP WebSocket 실시간 주문 현황
- Jenkins CI/CD 자동 배포 · GitHub Actions 구성 · HTTPS 적용

PROJECT
TYPE

팀 프로젝트 (3인)

PERIOD

2025.12.22 — 2026.01.30 (약 6주)

MY ROLE

영화 조회 · 매점 · 주문 · 결제 · 실시간 · 인증 · CI/CD

GITHUB

github.com/AllMovieProject/AllMovieProject

TECH STACK

Spring Boot 3.5

Vue.js

Pinia

MyBatis

Oracle

WebSocket

STOMP

Spring Security

Docker

Jenkins

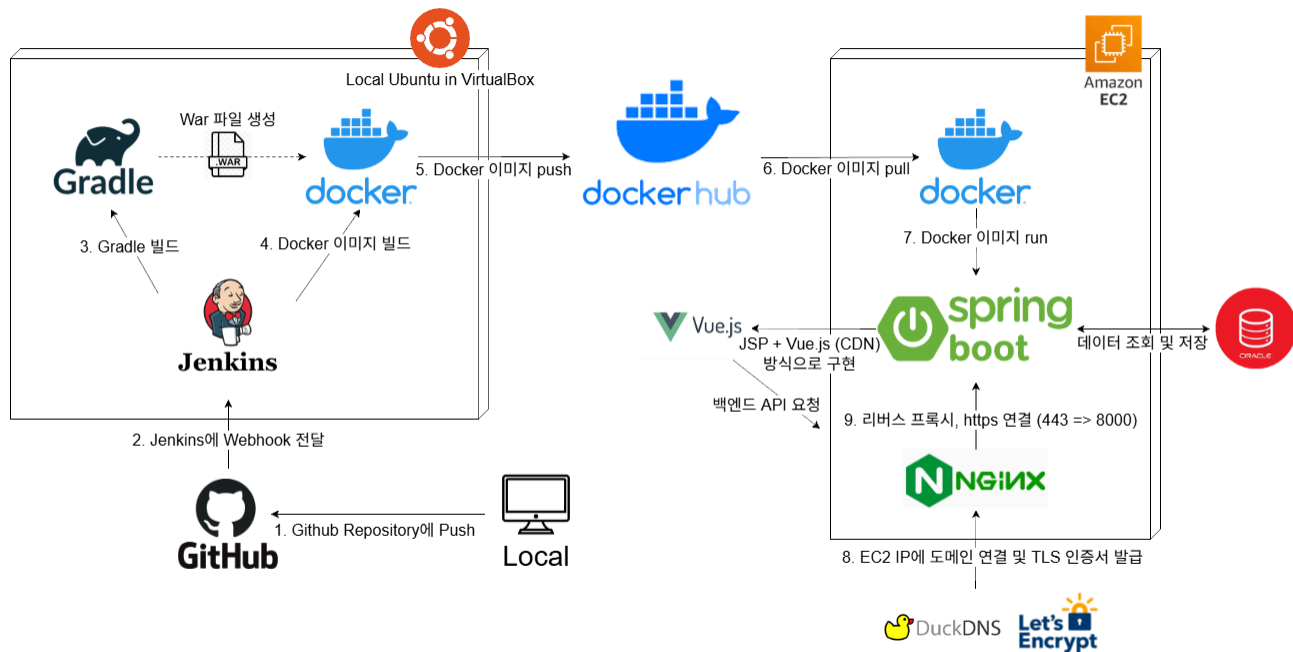
GitHub Actions

AWS EC2

HTTPS

포트원

시스템·배포 아키텍처





개발 목표 & 성과

개발 목표

MyBatis를 심화 적용해 복잡한 조건 검색과 다중 테이블 조회를 처리한다.

Spring Boot 기반으로 MVC 구조를 잡고 계층 간 역할을 분리한다.

PG를 연동해 결제 서비스를 구현한다.

Spring Security로 인증/인가를 구현하고 역할 기반 접근 제어를 적용한다.

Docker + Jenkins로 CI/CD 파이프라인을 구성해 자동화 배포를 구현한다.

성 과

제목 · 작성자 · 내용을 조합한 다중 조건 검색을 동적 쿼리로 구성했습니다. 주문 조회는 resultMap의 association · collection으로 묶어 단일 쿼리로 줄였습니다.

Controller · Service · Mapper 계층을 분리하고 @RestControllerAdvice로 예외 처리 지점을 한 곳에 모았습니다.

포트원 PG를 연동해 결제 요청부터 완료·취소까지의 흐름을 구현했습니다.

USER · MANAGER · ADMIN 세 역할로 접근 경로를 분리해 권한별 기능을 구분했습니다.

Gradle 빌드 · Docker 이미지 · AWS EC2 배포까지 자동화해 수동 배포 과정을 제거했습니다.

프로젝트 둘러보기

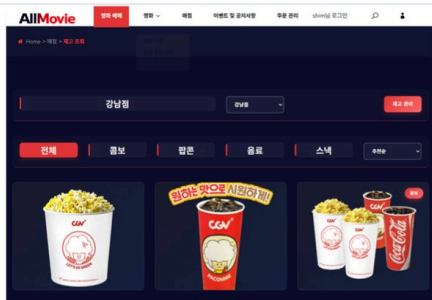


01 메인 홈



02 영화 예매

팀원



03 매점



04 커뮤니티

팀원

담당 기능 — 영화 정보 조회

요구사항분석표

기능	연결 페이지	요소	동작	행위자	우선순위
영화 목록 조회	영화 상세	포스터, 제목, 장르, 제작연도, 페이지네이션	영화 목록 12개씩 페이지네이션 · 포스터 · 제목 클릭 시 상세 이동	사용자	상
영화 상세 조회	영화 목록	제목, 장르, 포스터, 상영시간, 감독, 주연, 줄거리, 등급, 개봉일	선택한 영화의 상세 정보 출력	사용자	상
영화 리뷰	-	리뷰 작성 버튼, 작성자, 작성일, 작성량, 리뷰 목록	로그인 후 리뷰 작성 · 목록 출력	사용자	중

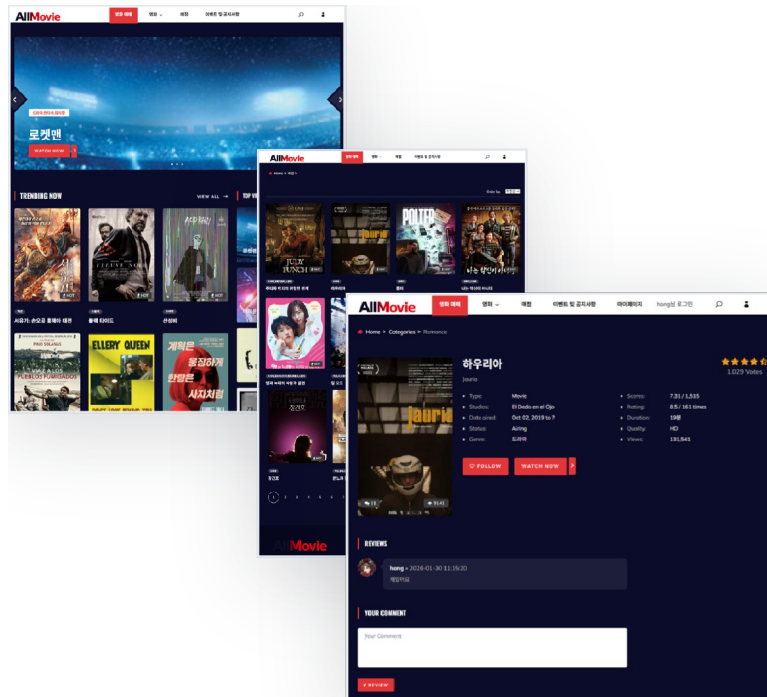
구현 방식

영화 목록 조회

영화를 12건 단위로 페이징 조회하고 전체 페이지 수를 함께 계산해 목록에 노출했습니다.

영화 상세 · 리뷰

선택한 영화의 상세 정보와 리뷰 목록을 조회하고, 로그인 사용자는 리뷰를 작성합니다.



담당 기능 — 매점

요구사항분석표

기능	연결 페이지	요소	동작	행위자	우선순위
상품 목록 조회	상품 상세 조회	극장 선택, 상품 유형 탭, 정렬(인기/등록/가격), 대표 이미지, 가격, 상품명	극장·유형별 조회 · 인기/등록/가격 정렬 · 상품 클릭 시 상세 이동	사용자	상
상품 상세 조회	상품 목록, 장바구니, 결제, 로그인	상품명, 대표 이미지, 가격, 목록 버튼, 상세 내용, 옵션 선택, 수량 버튼, 장바구니 버튼, 결제 버튼	옵션·수량별 가격 변동 · 비로그인 시 로그인 유도 · 결제/장바구니 진입	사용자	상
장바구니	상품 상세, 상품 결제	상품명, 대표 이미지, 가격, 목록 버튼, 옵션 선택, 수량 버튼, 결제 버튼	담은 상품 조회 · 옵션·수량 조정 · 결제 진입 · 상품명 클릭 시 상세	사용자	상

구현 방식

상품 목록 조회

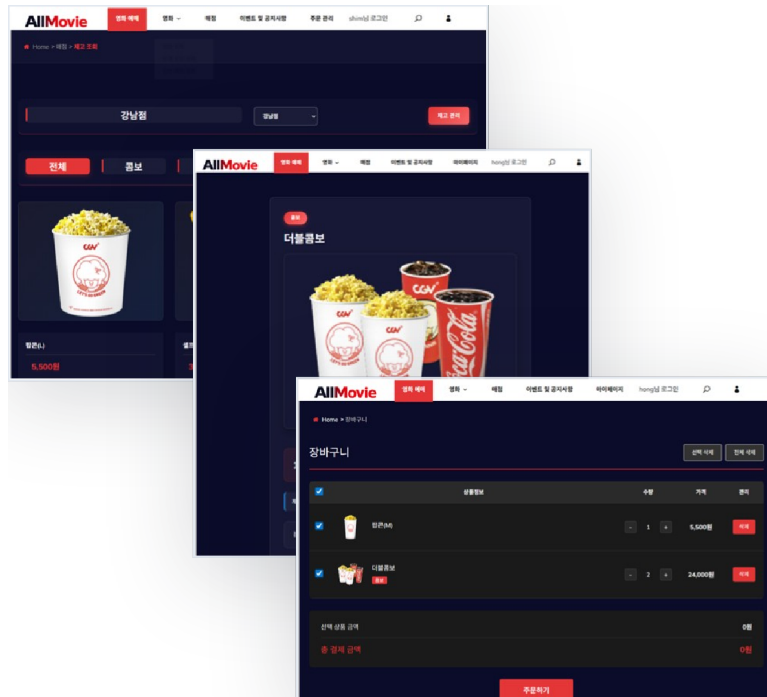
극장별 재고를 상품 정보와 **조인**해 등록일순으로 조회하고, 카테고리·콤보 필터와 이름·가격 정렬은 화면에서 처리했습니다.

상품 상세

옵션별 추가 금액을 상품 정보와 함께 조회하고, 옵션과 수량을 고를 때마다 가격을 계산합니다.

장바구니

담은 상품의 옵션과 수량을 조정하고 결제로 넘어갑니다.



담당 기능 — 주문 · 결제 · 마이페이지

요구사항분석표

기능	연결 페이지	요소	동작	행위자	우선순위
상품 주문 결제	상품 목록	상품명, 대표 이미지, 가격, 취소 버튼, 결제 버튼	옵션·수량에 따라 주문 결제·완료 시 목록 이동·취소 시 이전 페이지	사용자	상
매점 주문 현황 조회	상품 목록 조회	상품명, 주문 시각, 주문 상태, 극장 위치	접수/거절/준비중/준비완료 실시간 표시	사용자	상
상품 주문 내역 조회	마이페이지	주문 결과, 주문 메뉴 정보(상품명·옵션·가격), 결제 금액, 극장 위치	주문 목록 조회·픽업완료/취소 결과 표시	사용자	상

구현 방식

주문 결제

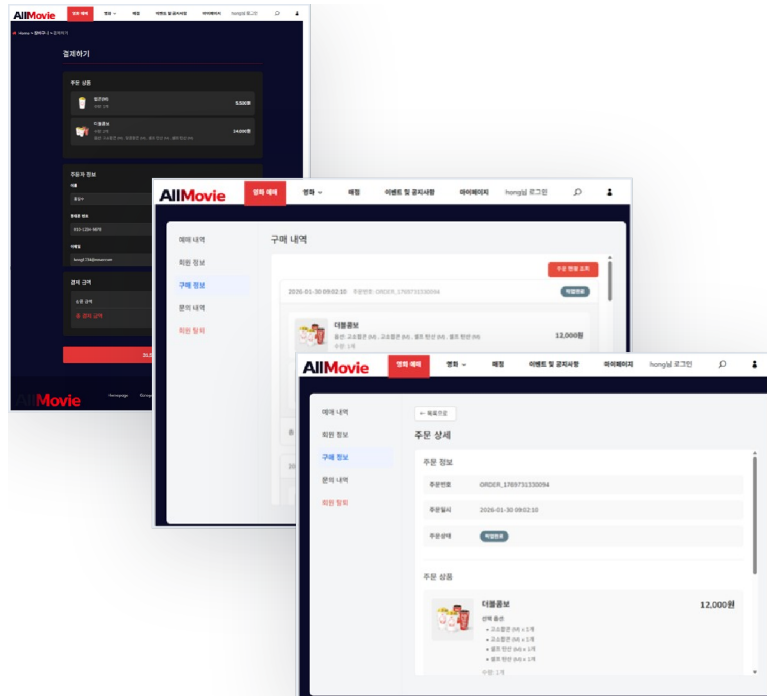
결제 요청 시 장바구니를 서버에서 조회해 **주문·주문항목·옵션**을 생성하고, **포트원** 결제 결과를 받아 상태를 기록했습니다.

실시간 주문 현황

주문 상태가 바뀌면 사용자에게 알림을 보내고, 마이페이지 주문 목록도 함께 갱신합니다.

주문 내역

주문 메뉴·옵션·결제 금액과 픽업 결과를 조회합니다.



담당 기능 — 관리자 (주문 상태 · 재고)

요구사항분석표

기능	연결 페이지	요소	동작	행위자	우선순위
주문 상태 전송	관리자 페이지	주문 메뉴 정보(사용자·상품·옵션·금액·위치), 주문 상태(접수/거절/준비중/준비완료) 버튼	주문 발생 시 접수·거절·거절 시 환불·준비 완료 알림 전송	직원	상
상품 재고 관리	관리자 페이지	상품명, 대표 이미지, 가격, 재고, 재고 추가 폼, 저장 버튼	상품 재고 조회·추가·수정	직원	중

구현 방식

주문 목록 조회

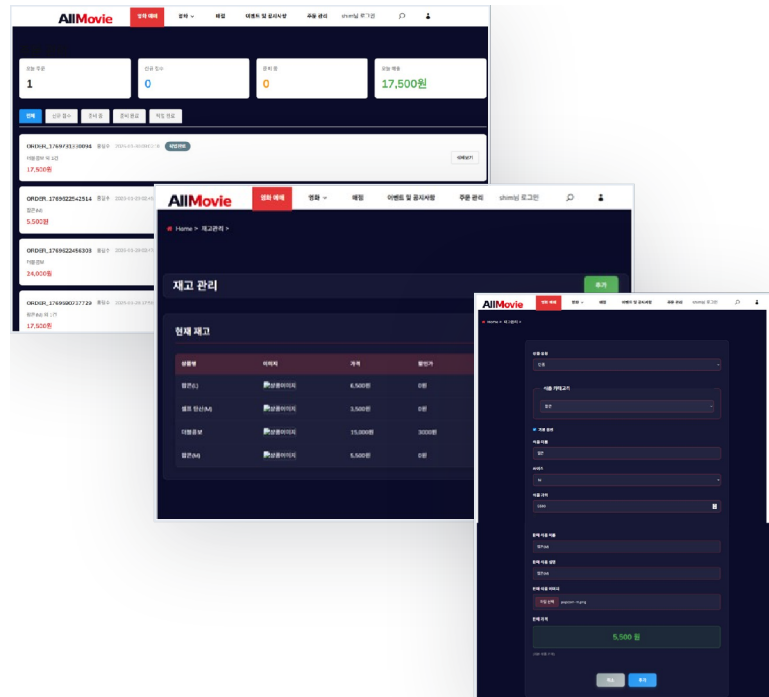
주문과 주문 항목·옵션을 조회합니다.

주문 상태 전송

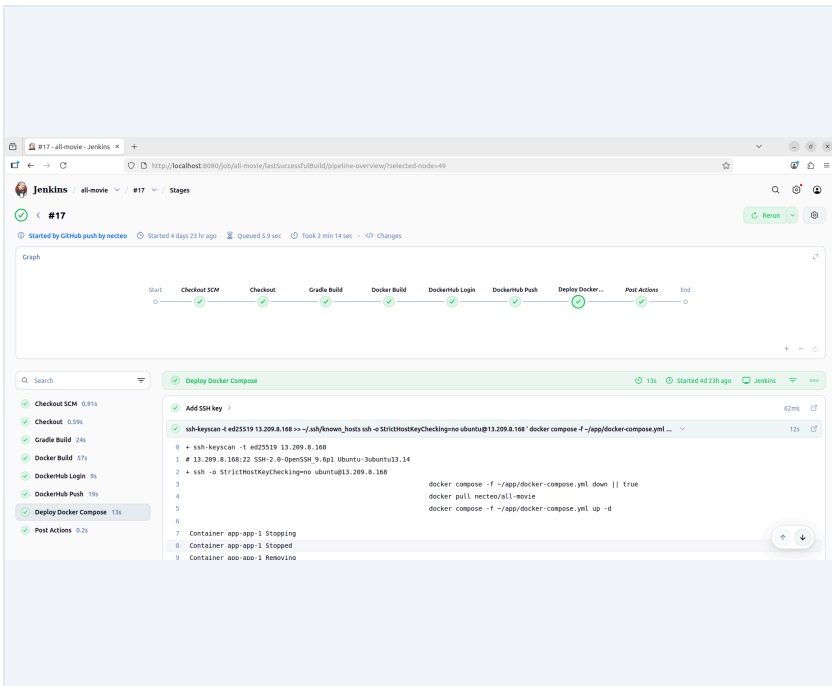
직원이 주문을 접수·거절하고 상태를 바꾸면 준비완료 알림을 STOMP WebSocket으로 브로드캐스트합니다.

재고 관리

직원 계정이 담당하는 극장의 재고 목록을 조회하고 상품별 재고를 등록합니다.

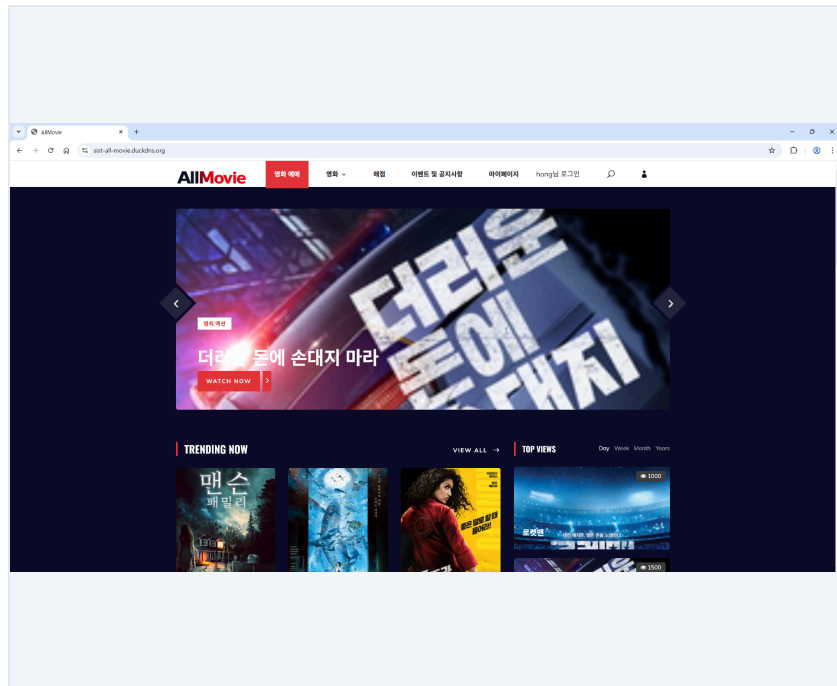


파이프라인 & HTTPS 배포



The screenshot shows the Jenkins web interface for a pipeline named 'all-movie'. The pipeline is in a successful state, indicated by a green checkmark and the label '#17'. The pipeline graph shows a sequence of steps: Start, Checkout SCM, Checkout, Gradle Build, Docker Build, DockerHub Login, DockerHub Push, Deploy Docker..., and Post Actions. The 'Deploy Docker Compose' step is currently selected, showing its configuration and logs. The logs indicate that the Docker Compose file was pulled and the containers were started successfully.

01 Jenkins 파이프라인 전 단계 성공



The screenshot shows the AllMovie website. The header includes the AllMovie logo and navigation links. The main banner features a movie poster for '더리온' (The Lion) with the text '더리온 돈에 손대지 마라' (Don't touch the money of The Lion). Below the banner, there is a 'TRENDING NOW' section displaying several movie posters, including '맨스 페밀리' (Men's Family) and '올리비아' (Olivia).

02 AWS 배포 + HTTPS 적용

트러블슈팅 — 실시간 · CI/CD

01 · 실시간 주문 현황

▲ 문제

1초 폴링이 만든 불필요한 DB 조회

매점 주문 현황을 1초 간격 폴링으로 구현했으나, 불필요한 DB 조회가 접속자 수에 비례해 늘어나는 구조였습니다.

✅ 해결

STOMP WebSocket · 이벤트 드리븐 통신

WebSocket(STOMP)으로 바꿔 주문 이벤트가 생길 때만 갱신 신호를 보냅니다. 화면은 신호를 받은 순간에만 조회하고, 주문이 없으면 요청도 없습니다.

```
// 직원 화면에는 갱신 신호만
messagingTemplate.convertAndSend("/topic/orders/" + store_id, "update");

// 사용자에게는 주문 상태를 담아 전송
messagingTemplate.convertAndSend("/topic/user/orders", userNotification);
```

02 · CI/CD 파이프라인

▲ 문제

반복되는 수동 배포

매 배포마다 서버에 접속해 git clone으로 소스를 받고, Gradle 빌드와 java -jar 실행을 직접 해야 했습니다.

✅ 해결

Jenkins 파이프라인으로 배포 자동화

Git 체크아웃부터 Gradle 빌드 → Docker 이미지 빌드 → DockerHub Push → AWS EC2 원격 배포까지 6단계를 파이프라인으로 묶었습니다. GitHub에 push하면 자동으로 배포됩니다.

프로젝트 개요

PROJECT 03 · SOLO

BookTalk

책을 좋아하는 사용자가 도서 정보를 탐색하고, 게시판에서 소통하며, AI 챗봇에게 책을 추천받는 풀스택 SPA입니다. **Spring Boot**와 **Express**로 백엔드를 나누고, **React SPA**가 두 서버를 각각 호출합니다.

핵심 기능

- 도서 목록 · 상세 조회 / 커뮤니티 게시판
- Google · Kakao · Naver 소셜 로그인 (OAuth2)
- JWT HttpOnly 쿠키 · Access(15m) / Refresh(7d) Rotation
- Node.js/Express 서버 분리 · YouTube / 뉴스 연동
- Spring AI + Gemini 챗봇

PROJECT TYPE

개인 프로젝트

PERIOD

2026.02.12 — 2026.06.14

MY ROLE

기획 · 설계 · 개발 전 과정

GITHUB

github.com/necteo/BookTalk-spring
github.com/necteo/BookTalk-react

TECH STACK

Spring Boot 3.x

React 19

TypeScript

React Query

JPA

MySQL

JWT

OAuth2

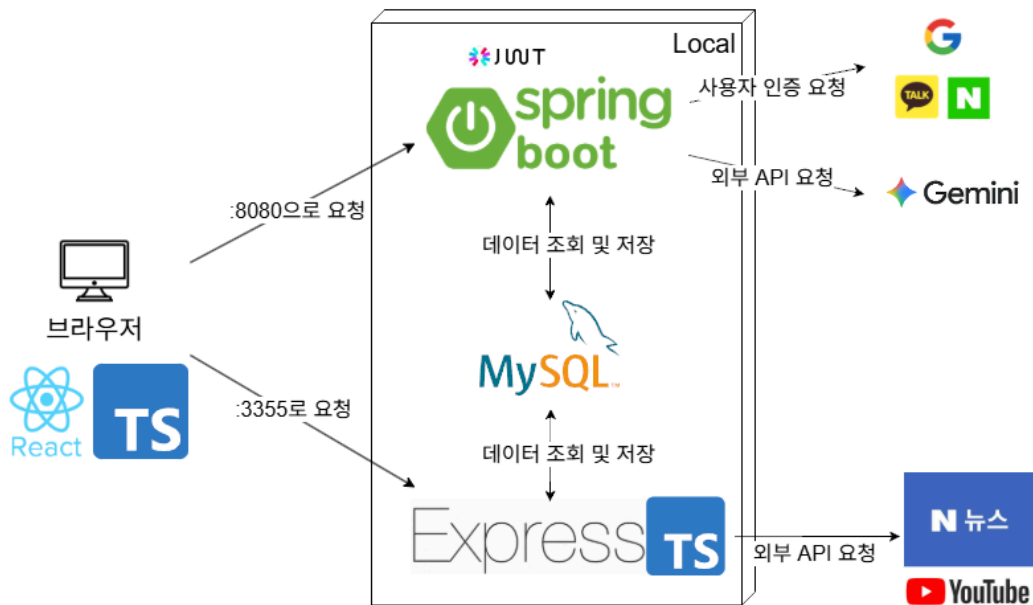
Spring AI

Gemini

Node.js

Express

시스템 아키텍처



개발 목표 & 성과

개발 목표

Spring Boot를 API 서버로, React를 독립 SPA로 분리해 풀스택 구조를 구현한다.

JWT와 Google · Kakao · Naver 소셜 로그인을 연동해 인증/인가 흐름을 구성한다.

TanStack Query를 활용해 서버 상태 관리와 캐싱 구조를 적용한다.

Spring Boot 외에 Node.js · Express를 별도 서버로 두어 커뮤니티 게시판과 외부 API 프록시를 분리 구성한다.

성 과

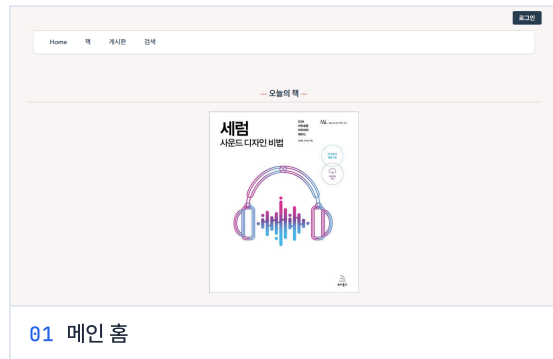
Spring Boot는 REST API 서버로, React는 독립 SPA로 나뉘었습니다. origin이 다르므로 통신은 CORS로 열었습니다.

Access/Refresh Token 발급과 갱신, 소셜 로그인 콜백 처리를 구현했습니다.

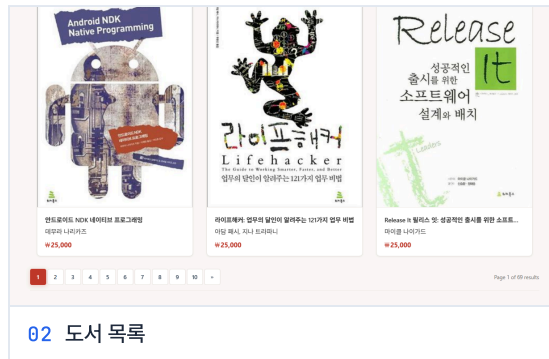
TanStack Query로 서버 데이터를 캐싱해 중복 요청을 줄이고, 변경 후에는 refetch로 다시 불러옵니다.

Express 서버에 게시판 CRUD와 외부 API 프록시를 올리고, API 키는 서버 환경 변수에만 뒀습니다.

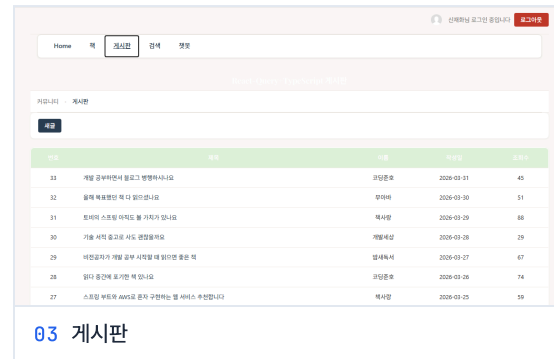
프로젝트 둘러보기



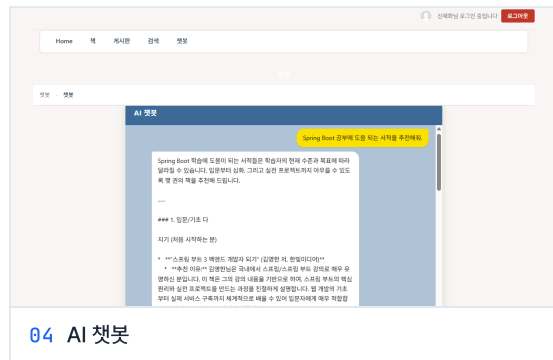
01 메인 홈



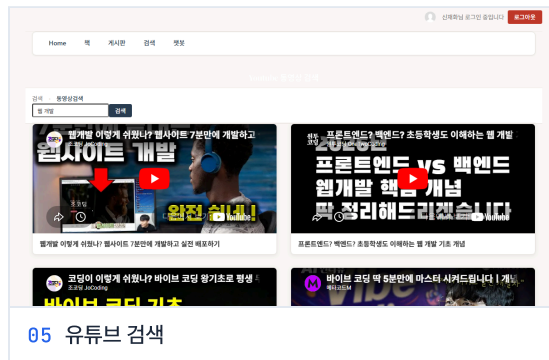
02 도서 목록



03 게스트



04 AI 챗봇



05 유튜브 검색



06 뉴스 검색

담당 기능 — 도서 목록 · 상세 조회

구현 방식

메인 홈

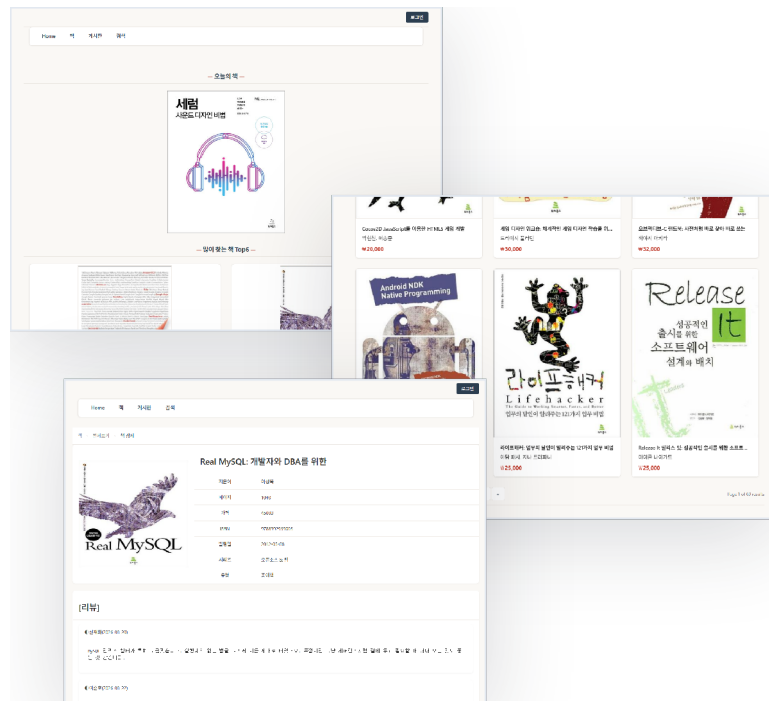
대표 도서와 신간·인기 목록을 한 응답에 묶어 내려줍니다. 신간은 출간일, 인기는 조회수 기준으로 정렬하고 **Pageable**로 노출 개수를 제한했습니다.

도서 정보 · 리뷰

도서 상세 정보와 리뷰 목록을 조회하고, 리뷰 작성·수정·삭제를 처리합니다.

페이징

목록을 **페이지당 12건씩** 서버에서 나누고, **10개 단위 블록**으로 출력했습니다.



담당 기능 — 게시판

구현 방식

서버 구성

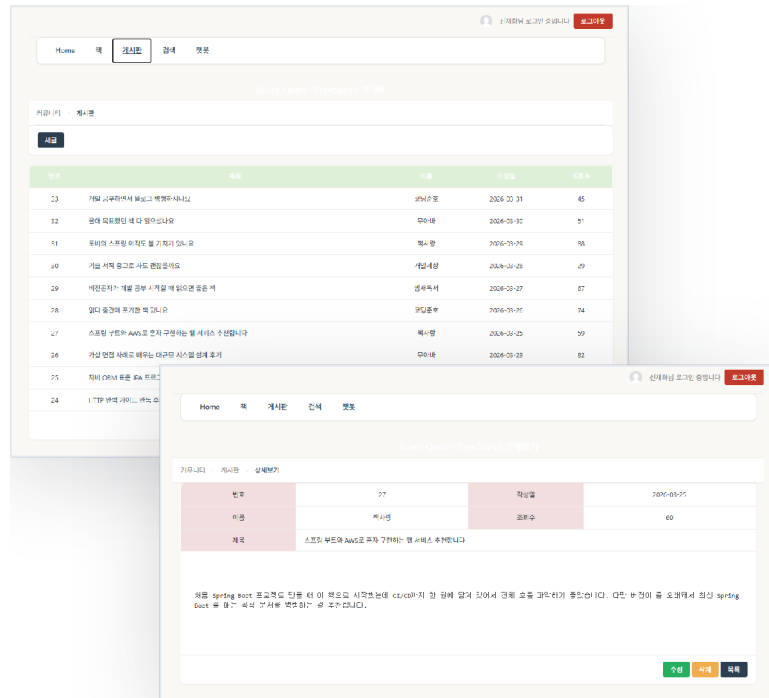
게시판은 별도 **Express** 서버로 구성하고 **MySQL**과 연동했습니다.

프론트 연동

게시판과 유튜브·뉴스는 **Express**, 나머지는 **Spring Boot**를 호출하도록 **Axios** 인스턴스를 나눴습니다.

게시글 CRUD

글 작성 시 지정한 **비밀번호**를 서버에서 대조해 수정·삭제를 허용하고, **React** 폼과 **REST API**를 연결했습니다.



담당 기능 — 소셜 로그인 · 인증

구현 방식

OAuth2 연동

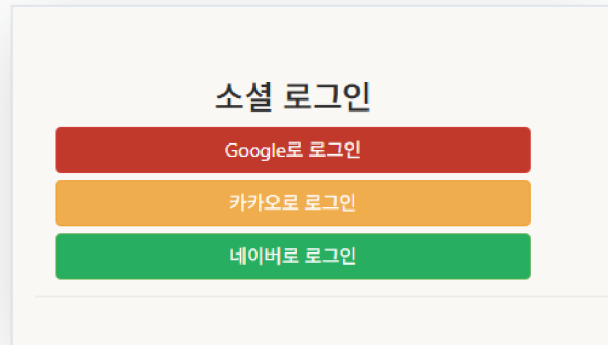
Google·Kakao·Naver 3사 OAuth2를 연동하고 provider마다 다른 사용자 정보 응답을 **Factory**로 분기했습니다.

토큰 관리

토큰은 둘 다 **HttpOnly** 쿠키에 담고 전송 경로를 나눴습니다. Access는 전체 경로, Refresh는 `/api/auth`에만 실립니다.

자동 재발급

Axios 인터셉터가 401을 감지하면 토큰을 재발급하고 원래 요청을 다시 보냅니다. 재발급 요청 자체는 재시도에서 빼 무한 루프를 막았습니다.



담당 기능 — AI 챗봇

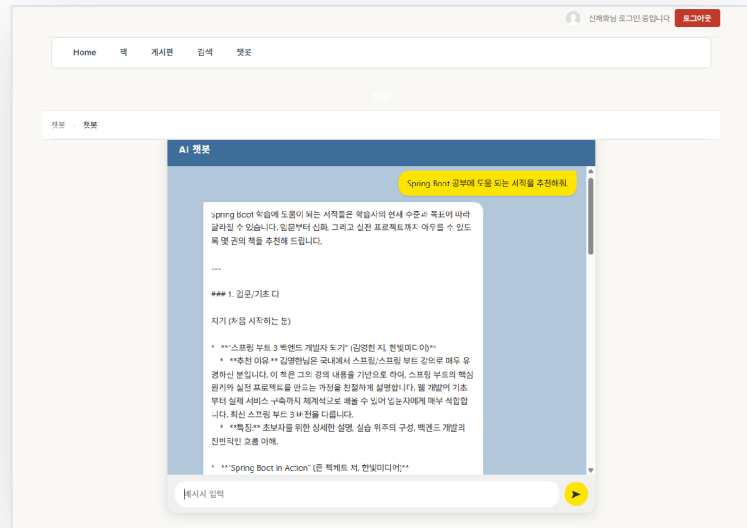
구현 방식

Gemini 연동

Spring AI로 Gemini를 연동하고 SSE로 응답을 스트리밍했습니다.

문장 단위 스트리밍

토큰 단위로 내보내면 화면이 끊겨 보여서, 마침표와 한국어 종결어미를 기준으로 문장 단위로 묶어 보냅니다.



담당 기능 — 유튜브 · 뉴스 조회

구현 방식

검색 페이지

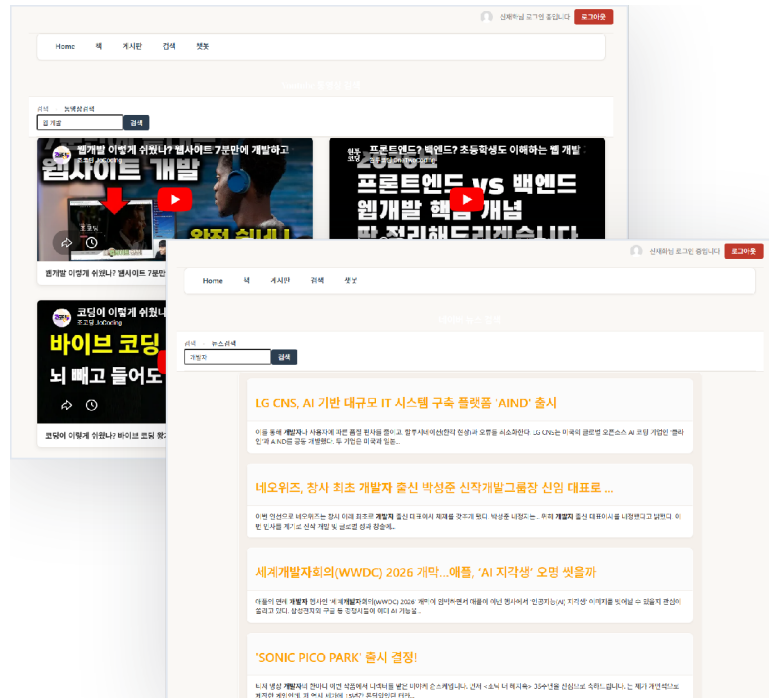
유튜브와 뉴스를 각각 별도 페이지로 두고 입력한 검색어로 조회합니다.

검색 결과 조회

유튜브는 영상 28건, 뉴스는 기사 50건을 받아 목록으로 보여줍니다.

검색어 캐싱

검색어를 `queryKey`에 담아 같은 검색어는 캐시에서 바로 꺼냅니다.



트러블슈팅 — API 키 · 접근 제어

01 · API 키 은닉

▲ 문제

번들에 그대로 실려 나간 YouTube API 키

YouTube 검색을 React에서 직접 호출하며 **API 키를 코드에 하드코딩**했습니다. 키가 빌드 번들 · 네트워크 요청에 그대로 실려 개발자도구로 확인할 수 있었고, git 히스토리에도 남았습니다.

```
const API_KEY = "AIzaSy*****";
// 브라우저 번들 · git에 그대로 노출
```

✓ 해결

외부 API 호출을 Express 프록시로 이전

호출 책임을 **Express 서버**로 옮기고 키는 서버 .env에만 보관했습니다. 프론트는 자기 백엔드만 호출하므로 키가 클라이언트로 전달되지 않습니다. 하드코딩돼 있던 네이버 키까지 함께 분리했습니다.

```
const key = process.env.YOUTUBE_API_KEY; // 서버에만
app.get('/youtube/find-node', async (req, res) => {
  const { data } = await axios.get('...&key=' + key);
  res.json(data); // 프론트는 이 엔드포인트만 호출
});
```

02 · 접근 제어 (인증 · 소유권 검증)

▲ 문제

작성자 클라이언트가 정하던 도서 리뷰 API

단순 로그인 시절 만든 도서 리뷰 API는 **작성자(id · name)**를 요청 본문에서 받았습니다. 소셜 로그인(OAuth2)으로 전환한 뒤에도 그 구조가 남아 있었습니다. 클라이언트가 보낸 값을 신뢰했기에 타인을 사칭해 리뷰를 쓰거나, 소유권 검증 없이 **리뷰 번호만으로 남의 리뷰를 삭제**할 수 있었습니다.

```
// Before - 작성자 · 소유권을 클라이언트 입력에 의존
commentInsert(@RequestBody CommentDTO dto) // dto에 작성자 id/name 포함
commentDelete(@PathVariable int no, String isbn)
// 인증 · 소유권 검증 없음
```

✓ 해결

작성자 · 소유권을 JWT로 서버가 결정

요청 본문에서 작성자 정보를 제거하고 **인증 토큰에서 사용자를 식별**하도록 바꿨습니다. 작성자는 서버가 JWT로 정하고, 삭제는 **토큰의 사용자가 리뷰 작성자일 때만** 수행됩니다.

```
// After - Authentication(JWT)에서 결정
commentInsert(@RequestBody CommentInsertRequest req,
Authentication authentication) {
  me = memberService.getMe(authentication); // 작성자 = 인증 사용자
  cService.commentInsert(req, me.id(), me.name());
}
```

THANK YOU FOR READING

읽어주셔서 감사합니다.

맡은 일을 끝까지 책임지는 개발자가 되겠습니다. 함께할 기회를 기다리겠습니다.

EMAIL

jaehwa06@naver.com

GITHUB

github.com/necteo

BLOG

necteo.github.io